

Nodejs hefte 1

Eksempel 1: Hei fra terminalen

En JavaScript-fil kjøres med `node filnavn.js`. Programmet kan få informasjon fra terminalen gjennom `process.argv`.

Lag `hilsen.js`:

```
// Kjør med: node hilsen.js Kari

const navn = process.argv[2] ?? "verden";

console.log(`Hei, ${navn}!`);
console.log("Du kjører Node-versjon", process.version);
console.log("Alle argumentene:", process.argv);
```

Kjør `node hilsen.js Kari`, og deretter `node hilsen.js` uten navn.

Se på utskriften av `process.argv`. Plass 0 er stien til Node, plass 1 er stien til filen, og det du skriver selv starter på plass 2. `??` gir en standardverdi når ingenting ble skrevet.

Ekstraoppgaver:

1. Kjør `node hilsen.js Ola Nordmann` og deretter `node hilsen.js "Ola Nordmann"`. Hva er forskjellen, og hvorfor?
2. Ta imot en alder som argument nummer to, og skriv ut hvor gammel personen blir om 10 år. Hva skjer hvis du glemmer `Number()`?
3. Skriv navnet med store bokstaver og hvor mange bokstaver det har.
4. Legg til `document.title = "Hei"` nederst. Les feilmeldingen: hvilken linje peker den på, og hvorfor finnes ikke `document` i Node?

Eksempel 2: Egne moduler

Kode kan deles i flere filer. Det som skal brukes utenfor filen må ha `export` foran seg, og hentes inn med `import`.

Kjør først disse to kommandoene i mappa di. Den andre sier til Node at vi bruker `import` og `export`:

```
npm init -y
npm pkg set type=module
```

Lag `matte.js`:

```
export function pluss(a, b) {
  return a + b;
}

export function gange(a, b) {
  return a * b;
}

export const PI = 3.14159;

// Ikke eksportert, så den er "privat" for denne filen
function hemmelig() {
  return "Denne ser ingen andre filer";
}
```

Lag `kalkulator.js`:

```
// Kjør med: node kalkulator.js 6 7

import { pluss, gange, PI } from "./matte.js";

const a = Number(process.argv[2]);
const b = Number(process.argv[3]);

console.log(`${a} + ${b} = ${pluss(a, b)}`);
console.log(`${a} * ${b} = ${gange(a, b)}`);
console.log(`Arealet av en sirkel med radius ${a} er ${PI * a * a}`);
```

Legg merke til to ting i importen: `./` betyr "en fil i samme mappe", og `.js` må være med. Legg også merke til at arealet blir `113.09723999999999`. Datamaskiner regner ikke alltid helt nøyaktig med desimaltall.

Ekstraoppgaver:

1. Legg til `minus` og `dele` i `matte.js`, og bruk dem i kalkulatoren.
2. Prøv å importere `hemmelig`. Hva sier feilmeldingen?
3. Fjern `.js` fra importen. Hva sier feilmeldingen nå?
4. Kjør `node kalkulator.js hei 3`. Sjekk med `Number.isNaN()` om tallene er gyldige, og skriv en vennlig feilmelding hvis de ikke er det.
5. Vis arealet med to desimaler ved hjelp av `.toFixed(2)`.

Eksempel 3: npm, pakker og skript

Med npm kan du hente ferdig kode som andre har skrevet. Filen `package.json` beskriver prosjektet ditt: navn, hvilke pakker det bruker og hvilke skript som finnes.

Installer en pakke som gir farger i terminalen:

```
npm install chalk
```

Åpne `package.json`. Der har `chalk` dukket opp under `dependencies`. Du har også fått en mappe `node_modules` og en fil `package-lock.json`. Mappa `node_modules` skal aldri deles eller lastes opp, fordi `npm install` kan lage den på nytt ut fra `package.json`.

Lag `farger.js`:

```
import chalk from "chalk";

const klokka = new Date().toLocaleTimeString("nb-NO");

console.log(chalk.green("Alt gikk bra!"));
console.log(chalk.red.bold("Feil: noe gikk galt"));
console.log(chalk.blue(`Klokka er ${klokka}`));
console.log(chalk.bgYellow.black(" Advarsel ") + " husk å lagre filen");
```

Legg til to skript i `package.json`, enten for hånd under `"scripts"` eller med denne kommandoen:

```
npm pkg set scripts.start="node farger.js" scripts.dev="node --watch farger.js"
```

Kjør `npm start`. Kjør så `npm run dev`, endre en tekst i filen og lagre: programmet kjører på nytt av seg selv. Avslutt med Ctrl+C. Slik kommer du til å jobbe når vi lager webservere med Express.

Ekstraoppgaver:

1. Slett hele `node_modules`-mappa og kjør `npm start`. Hva skjer? Kjør `npm install` og prøv igjen.
2. Prøv andre farger og stiler, for eksempel `chalk.magenta.underline`.
3. Installer pakken `figlet` og skriv ut navnet ditt med store ASCII-bokstaver: `import figlet from "figlet"` og `console.log(figlet.textSync("Kari"))`.
4. Lag et eget skript `"hilsen": "node hilsen.js"` og kjør det med `npm run hilsen`.

Eksempel 4: Lese og skrive filer

Node kan lese og skrive filer på maskinen din. Det kan ikke JavaScript i nettleseren.

Lag `handleliste.txt` med én vare per linje. Den tomme linja er med med vilje:

```
melk
brød
egg
ost
bananer
```

appelsinjuice

Lag `handleliste.js`:

```
import { readFileSync, writeFileSync, appendFileSync } from "node:fs";

// 1. Les hele filen som tekst
const tekst = readFileSync("handleliste.txt", "utf-8");

// 2. Gjør teksten om til en array med én vare per element
const varer = tekst
  .split("\n")
  .map((linje) => linje.trim()) // fjerner mellomrom (og \r på Windows)
  .filter((linje) => linje !== ""); // fjerner tomme linjer

console.log(`Du har ${varer.length} varer på lista:`);
varer.forEach((vare, i) => console.log(`${i + 1}. ${vare}`));

// 3. Skriv en sortert kopi til en ny fil (overskriver hvis den finnes)
const sortert = [...varer].sort((a, b) => a.localeCompare(b, "nb"));
writeFileSync("sortert.txt", sortert.join("\n"));
console.log("Lagret sortert liste i sortert.txt");

// 4. Legg til en linje nederst i en loggfil (lager filen hvis den mangler)
appendFileSync("logg.txt", `${new Date().toISOString()} Leste ${varer.length} varer\n`);
```

Kjør programmet tre ganger og åpne `logg.txt` og `sortert.txt`. Loggen vokser for hver gang, mens den sorterte lista blir skrevet over. Det er forskjellen på `append` og `write`.

Ekstraoppgaver:

1. Legg til «ærter», «ørret» og «åkerbær» på lista. Sorteres de riktig? Fjern `"nb"` fra `localeCompare` og se hva som skjer.
2. Endre filnavnet i koden til en fil som ikke finnes. Les feilmeldingen, og fang den med `try/catch` slik at programmet skriver en vennlig melding i stedet.
3. La programmet ta imot en ny vare fra terminalen (`node handleliste.js sjokolade`) og legge den til i `handleliste.txt` med `appendFileSync`.
4. Lagre varene som JSON i `handleliste.json` med `JSON.stringify(varer, null, 2)`. Åpne filen og se hvordan den ser ut.

Eksempel 5: Asynkron kode

Noen ting tar tid, for eksempel å lese en fil eller hente data fra en database. Node venter ikke på dem, men fortsetter med resten av koden og kommer tilbake når de er ferdige.

Del A: Lag `rekkefolge.js`. Gjett i hvilken rekkefølge linjene skrives ut før du kjører programmet.

```
import { readFile } from "node:fs";

console.log("1. Før vi leser filen");

readFile("handleliste.txt", "utf-8", (feil, tekst) => {
  console.log("3. Filen er lest, den har", tekst.length, "tegn");
});

console.log("2. Etter at vi ba om å lese filen");
```

Kjør programmet. Ble det som du trodde? Funksjonen som sendes til `readFile` kalles en *callback*: «ring meg når du er ferdig». Med mange slike inni hverandre blir koden fort uoversiktlig.

Del B: Den moderne måten med promises og `await`. Lag `vent.js`:

```
import { readFile } from "node:fs/promises";

// Returnerer et promise som blir ferdig etter ms millisekunder
function vent(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

console.log("Starter...");

const tekst = await readFile("handleliste.txt", "utf-8");
console.log("Første linje i handlelista:", tekst.split("\n")[0]);

await vent(1000);
console.log("Ett sekund senere");

await vent(1000);
console.log("To sekunder senere. Ferdig!");
```

Nå kan koden leses ovenfra og ned, selv om den er asynkron. Et *promise* er et løfte om et svar senere, og `await` betyr «vent her til løftet er innfridd». Legg merke til at vi nå importerer fra `node:fs/promises`.

Ekstraoppgaver:

1. Lag en nedtelling fra 5 til 1 med `vent(1000)` i en `for`-løkke, og skriv «Lunsj!» til slutt.
2. Lag en `async function lesListe(filnavn)` som leser en fil og returnerer en array med varer. Kall den med `await`. Hva får du hvis du glemmer `await`?
3. Les en fil som ikke finnes med `await readFile(...)`, og fang feilen med `try/catch`.
4. Utfordring: mål tiden med `console.time("tid")` og `console.timeEnd("tid")`. Kjør først tre `await vent(1000)` etter hverandre, og deretter `await Promise.all([vent(1000), vent(1000), vent(1000)])`. Hvorfor tar den andre kortere tid?

Oppgaver

Oppgavene blir vanskeligere utover. Lag en ny mappe `oppgaver`, og kjør `npm init -y` og `npm pkg set type=module` der før du begynner.

Oppgave 1: Alder og fødselsår

Lag `alder.js` som tar imot et navn og en alder fra terminalen:

```
node alder.js Kari 17
```

Programmet skal skrive ut:

```
Hei, Kari! Du er 17 år.  
Du er født i 2009 eller 2008.  
Om 10 år er du 27 år, og da er det år 2036.
```

- Bruk `new Date().getFullYear()` for å finne årstallet, ikke skriv det inn selv.
- Hvis navn eller alder mangler, skal programmet forklare hvordan det skal brukes, i stedet for å skrive `NaN`.

Oppgave 2: Tekstverktøy-modul

Lag en modul `tekstverktoy.js` med tre eksporterte funksjoner:

- `antallOrd(tekst)` returnerer antall ord
- `baklengs(tekst)` returnerer teksten baklengs
- `erPalindrom(tekst)` returnerer `true` hvis teksten er lik forfra og bakfra (store og små bokstaver og mellomrom teller ikke)

Lag `tekst.js` som importerer funksjonene og bruker dem på teksten fra terminalen:

```
node tekst.js Agnes i senga
```

```
Tekst: Agnes i senga  
Antall ord: 3  
Baklengs: agnes i sengA  
Det er et palindrom!
```

Hint: `process.argv.slice(2).join(" ")` setter sammen alle ordene. For å snu en tekst kan du bruke `split("")`, `reverse()` og `join("")`.

Oppgave 3: Terningspill med npm

Lag et nytt prosjekt i en egen mappe `terning`:

1. Kjør `npm init -y`, sett `type` til `module` og installer `chalk`.
2. Lag `terning.js` med en funksjon `kastTerning()` som returnerer et tilfeldig tall fra 1 til 6.
3. Kast to terninger og skriv ut begge og summen. Antall kast skal kunne gis i terminalen (`node terning.js 5`). Hvis ingenting er gitt, kastes det én gang.
4. Bruk farger: gul og «PAR!» når terningene er like, grønn når summen er 10 eller mer, rød når summen er 4 eller mindre.
5. Legg til skriptene `start` og `dev` (med `--watch`) i `package.json`, og test begge.

Hint: `Math.floor(Math.random() * 6) + 1`.

Oppgave 4: Gruppegenerator

Lag `elever.txt` med 10–15 navn, ett per linje. Lag `grupper.js` som:

1. leser navnene fra filen og fjerner tomme linjer
2. stokker navnene i tilfeldig rekkefølge
3. deler dem i grupper med størrelsen som gis i terminalen (`node grupper.js 3`)
4. skriver gruppene til skjermen og til filen `grupper.txt`

```
Gruppe 1: Nora, Kaja, Bente
Gruppe 2: David, Guro, Camilla
...
```

Hint for stokking: gå bakover gjennom arrayen og bytt hvert element med et tilfeldig element før det (Fisher–Yates). `slice` er nyttig for å dele opp i grupper.

Utfordring:

- Bruk `node:fs/promises` med `async/await` i stedet for de synkrone funksjonene.
- Med 14 elever i grupper på 3 blir siste gruppe ofte for liten. Fordel restelevene på de andre gruppene, slik at ingen gruppe får færre enn den ønskede størrelsen.
- Gi filen dagens dato i navnet, for eksempel `grupper-2026-10-06.txt`.
- Lagre gruppene også som JSON i `grupper.json`.
- Gi en vennlig feilmelding med `try/catch` hvis `elever.txt` ikke finnes.

Oppgave 5: Notatapp i terminalen

Lag en notatapp som lagrer notater i filen `notater.json`, og som brukes slik:

```
node notat.js legg-til "Kjøp melk"
node notat.js legg-til "Lever oppgave 3"
node notat.js liste
```

1. Kjøp melk
2. Lever oppgave 3

Krav:

- Lag en egen modul `lagring.js` med to funksjoner: `async lesNotater()` og `async lagreNotater(notater)`. Bruk `node:fs/promises`, `JSON.parse` og `JSON.stringify`.
- Hvis `notater.json` ikke finnes, skal `lesNotater()` returnere en tom array i stedet for å krasje.
- Hvert notat er et objekt med minst `id` og `tekst`.
- Ukjente kommandoer skal gi en oversikt over hvilke kommandoer som finnes.

Utfordring:

- Legg til `slett <nr>`. Pass på at et nytt notat ikke får samme id som et notat som finnes fra før etter en sletting.
- Lagre tidspunktet notatet ble laget, og vis det i lista med `toLocaleString("nb-NO")`.
- Legg til `sok <ord>` som viser notater som inneholder ordet, uansett store eller små bokstaver.
- Bruk `chalk` til grønne bekreftelser og røde feilmeldinger.
- Tenk over: notatfilen fungerer som en enkel database. Hva kan gå galt hvis to personer bruker appen samtidig, eller hvis det blir 100 000 notater?